

HOMEWORK ASSIGNMENT 1: FORMULATING AND SOLVING A DESIGN OPTIMIZATION PROBLEM

- **Learning outcome:** By completing this assignment, you will be able to formulate a constrained design optimization problem from first-principles physics, analyze it using monotonicity analysis to assess well-posedness, solve it numerically, and interpret how the optimum shifts as design parameters vary.
- You may choose to use either programming language (MATLAB or Python) for the parts where coding is required.
- Your submission should include: (i) a PDF report named <McGill_ID>_HW1.pdf explaining your solution approach, and (ii) either a published MATLAB script or a Python notebook.

1. Simple drone design optimization

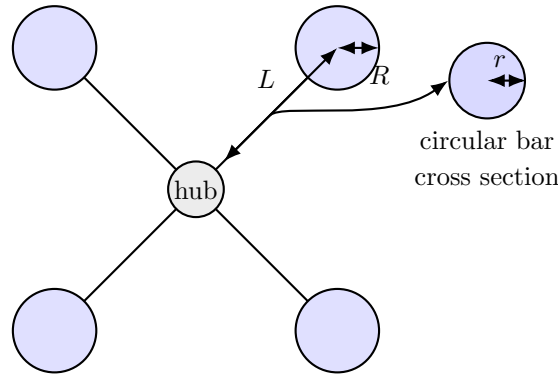


Figure 1: Simple quadrotor schematic (top view, not to scale): 4 identical arms of length L and fixed cross-section radius r radiate symmetrically from a central hub at a fixed inter-arm angle $\varphi = 90^\circ$, each carrying a rotor of radius R at its tip.

Consider a simple quadrotor with 4 identical arms radiating symmetrically from a central hub at a *fixed* inter-arm angle $\varphi = 90^\circ$, each carrying a rotor at its tip. To avoid the rotor blades overlapping, the propeller radius R and arm length L must satisfy a non-interference (clearance) condition: treating two adjacent arms and the hub as an isosceles triangle of side L and included angle φ , the chord between two adjacent rotor centers is $2L \sin(\varphi/2)$, so non-interference requires $2R \leq 2L \sin(\varphi/2)$, i.e.

$$R \leq L \sin(\varphi/2) = \frac{L}{\sqrt{2}}.$$

Assume each motor is rated for a fixed maximum electrical power P_0 , and that a power efficiency $\eta \in (0, 1]$ (lumping profile-drag, motor, and ESC losses) converts this into aerodynamic power, $P_i = \eta P_0$. Using *actuator-disk (momentum) theory*, the ideal aerodynamic power needed for a rotor of disk area $A = \pi R^2$ (air density ρ) to produce thrust T is

$$P_i = \frac{T^{3/2}}{\sqrt{2\rho A}} \implies T = (P_i \sqrt{2\rho A})^{2/3}.$$

Substituting $P_i = \eta P_0$ and $A = \pi R^2$, show that the thrust each rotor can produce, as a function of the rotor radius alone, is

$$T(R) = \left[\eta P_0 \sqrt{2\rho\pi} R \right]^{2/3}.$$

This thrust acts as a transverse tip load on the arm supporting that rotor. Model each arm as a cantilever beam of length L , fixed at the hub, with a *fixed* solid circular cross-section of radius r (second moment of area $I = \pi r^4/4$). Using elementary beam theory, show that the resulting maximum bending stress and tip deflection of the arm are

$$\sigma_{\max}(R, L) = \frac{T(R) L r}{I} = \frac{4 T(R) L}{\pi r^3}, \quad \delta(R, L) = \frac{T(R) L^3}{3EI} = \frac{4 T(R) L^3}{3\pi E r^4},$$

where E is the Young's modulus of the arm material. Finally, if the arm material has density ρ_{mat} , the combined mass of the 4 arms is

$$W(L) = 4 \rho_{\text{mat}} \pi r^2 L,$$

and the total drone mass is $m(L) = m_0 + W(L)$, where m_0 is the combined (fixed) mass of the hub, motors, propellers, battery, electronics, and payload.

- Select the rotor radius and the arm length as your two design variables, $x = (x_1, x_2) = (R, L)$ (fix all other parameters $r, \varphi, m_0, \rho, \rho_{\text{mat}}, E, P_0, \eta, g$). Using your own engineering judgment and a material of your choice, pick reasonable numerical values for these fixed parameters, for a maximum allowable stress σ_y and deflection $\delta_{\max}$, and for a required thrust margin (load) factor $k > 1$ such that the drone can still accelerate/maneuver at hover (i.e., the total available thrust must exceed the weight by a factor of k , $4T(R) \geq k m(L) g$). Formulate the optimization problem in *negative null form* to minimize the total arm mass W subject to *all four* constraints: $\sigma_{\max} \leq \sigma_y$, $\delta \leq \delta_{\max}$, the thrust-margin requirement, and the rotor-clearance condition.
- Before solving the problem, construct its *monotonicity table*: for each design variable ($x_1 = R$ and $x_2 = L$) and each function appearing in the formulation from part (a) (the objective W and each constraint $g_1, \dots, g_m$), determine whether that function is monotonically increasing (+), monotonically decreasing (−), or independent (*leave empty*) with respect to that variable. You may organize your table as follows:

	$x_1 = R$	$x_2 = L$
W		
g_1		
$\vdots$		
g_m		

Using this table, reason about whether the objective function is *bounded below* using only the constraints derived from the physics – i.e., for each variable, check whether at least one constraint becomes active in the direction that would otherwise let that variable be driven (toward 0 or ∞) to reduce the objective without limit. If any variable is left unbounded by the constraints alone, state what explicit variable bound(s) (e.g. $x_1 > 0$, $x_2 > 0$) must be added to the problem before it is well-posed.

- (c) Solve the problem formulated in part (a) (using code, e.g. a grid search or a nonlinear-programming solver such as MATLAB's `fmincon` or Python's `scipy.optimize.minimize`) to find the values of $x_1 = R$ and $x_2 = L$ that minimize the total arm mass W subject to *all four* constraints.
- (d) Now study how the optimum in part (c) changes with the problem parameters:
- Pick at least 3 different values of the rated motor power P_0 .
 - For each of these values, recompute the optimal (R, L) for at least 5 different values of the required thrust margin factor k .
- (e) (Bonus item) For each value of P_0 , plot the optimal arm mass W vs. the required thrust margin factor k . Optionally, also plot how the optimal R^* and L^* individually vary with k , and comment on whether the clearance constraint or the structural constraints (σ_y , $\delta_{\max}$) end up governing the optimum.

About MATLAB

Use the `help` option in MATLAB to find out about available MATLAB functions. The `help fmincon` command will give information on the `fmincon` function and what its inputs and outputs are.

About Python

Use the `help` option in Python to find out about available Python functions. The following command will give information on the `minimize` function and what its inputs and outputs are:

```
help scipy.optimize.minimize
```